

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 2

Meet the OS You Will Build (xv6)

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Setup

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
```

```
Cloning into '/home/sunflower/xv6-riscv'...
remote: Enumerating objects: 8334, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (22/22), done.
Receiving objects: 100% (8334/8334), 17.67 MiB | 2.23 MiB/s, done.
Resolving deltas: 100% (4583/4583), done.

Permissions Size User      Date Modified Name
drwxr-xr-x     - sunflower 28 Sep 17:30  kernel
drwxr-xr-x     - sunflower 28 Sep 17:30  mkfs
drwxr-xr-x     - sunflower 28 Sep 17:30  user
.rw-r--r--  1.2k sunflower 28 Sep 17:30  LICENSE
.rw-r--r--  5.7k sunflower 28 Sep 17:30  Makefile
.rw-r--r--  2.4k sunflower 28 Sep 17:30  README
.rwxr-xr-x  6.1k sunflower 28 Sep 17:30  test-xv6.py
```

First build (toolchain missing)

```
make qemu
```

```
*** Error: Couldn't find a riscv64 version of GCC/binutils.
*** To turn off this error, run 'gmake TOOLPREFIX= ...'.
/bin/sh: line 1: qemu-system-riscv64: command not found
/bin/sh: line 1: bc: command not found
gcc -march=rv64gc -g -ffile-prefix-map=/home/sunflower/xv6-riscv=. -c -o kernel/entry.
    o kernel/entry.S
cc1: error: bad value 'rv64gc' for '-march=' switch
make: *** [Makefile:99: kernel/entry.o] Error 1
```

Install toolchain (Arch Linux)

```
sudo pacman -S --needed base-devel git bc qemu-system-riscv \
     riscv64-elf-gcc riscv64-elf-binutils riscv64-elf-gdb
```

```
resolving dependencies...
looking for conflicting packages...
Total Download Size:   164.50 MiB
Total Installed Size:  994.16 MiB
:: Proceed with installation? [Y/n] y
```

Part 1 – Build and Boot

```
cd ~/xv6-riscv
make clean
make qemu
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format
    =raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$
```

Part 2 – Using xv6

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 38112
echo           2 4 36888
forktest       2 5 18248
grep           2 6 41648
init           2 7 37272
...
sync           2 22 36240
console        3 23 0

$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
... (truncated) ...

$ echo hello xv6
hello xv6

$ ls | grep c
cat            2 3 38112
echo           2 4 36888
wc             2 17 39136
wc: cannot open README240
console        3 23 0
xec 2u failed

$ wc README
48 336 2441 README

$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
...
```

```
test lazy_copy: OK
test lazy_copyinstr: OK
test lazy_sbrk: OK
test partial_write: OK
test unlinkcwd: OK
ALL TESTS PASSED
```

Part 3 – Reading the Source

```
$ sed -n 1,40p user/ls.c
```

```
#include kernel/types.h
#include kernel/stat.h
#include user/user.h
#include kernel/fs.h
#include kernel/fcntl.h

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, ls: cannot open %s\n, path);
    return;
  }

  if (fstat(fd, &st) < 0) {
```

```
$ sed -n 1,30p user/cat.c
```

```
#include kernel/types.h
#include kernel/fcntl.h
#include user/user.h

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, cat: write error\n);
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, cat: read error\n);
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
```

```
$ cat user/user.h
```

```
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
```

```
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
```

```
$ grep -n sys_read\|sys_write kernel/sysfile.c | head
```

```
69:sys_read(void)
83:sys_write(void)
```

Part 4 – sleep

Creating `user/sleep.c`:

```
#include kernel/types.h
#include user/user.h

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, usage: sleep <ticks>\n);
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Modifying `Makefile` to add `sleep` to `UPROGS`:

```
sed -i 's|^\t$U/_echo\$|&\n\t$U/_sleep\|' Makefile
grep -n _echo|_sleep Makefile
```

```
134:    $U/_echo
135:    $U/_sleep\
```

Running `sleep` in QEMU:

```
$ make qemu
$ sleep 10
$ sleep
```

```
usage: sleep <ticks>
```

Conclusion

In this laboratory work, I successfully set up and explored the xv6 operating system environment. By configuring the RISC-V toolchain on Arch Linux and compiling the kernel, I was able to boot the OS within the QEMU emulator. Throughout the session, I interacted with standard xv6 userland utilities, executed system tests, and analyzed key source files to understand how system calls and user programs are structured in C. Finally, I practically applied these concepts by writing a custom `sleep` user-space program, modifying the Makefile to include it in the build process, and successfully executing it within the running xv6 system. This provided a strong foundational understanding of xv6's architecture and the practical workflow for developing OS-level software.